

MODULE 5

Feature Engineering

From raw vibration windows to numbers that fit a compact model

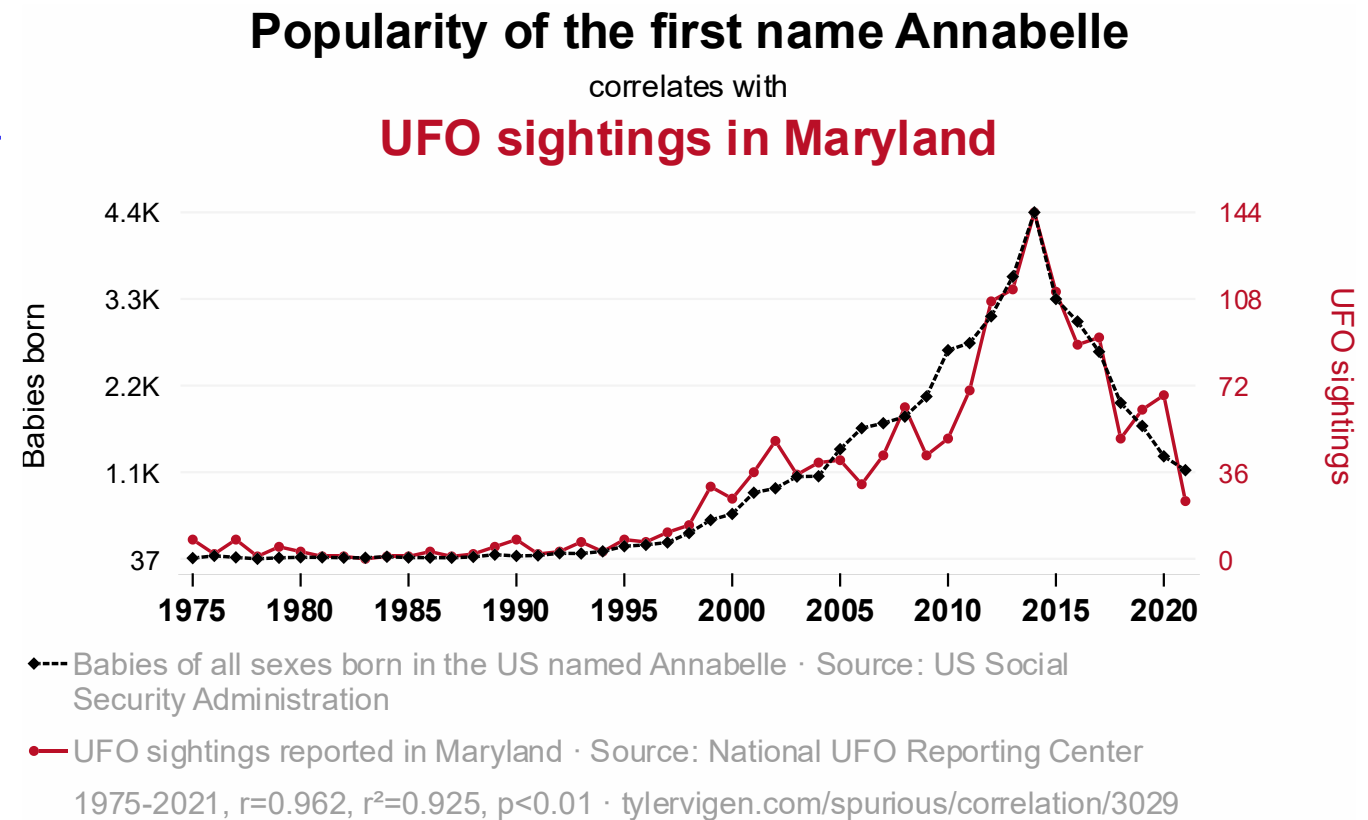
TinyML Course · Day 2

Why features at all?

- A 2 s window = 200 samples $\times$ 3 axes = **600 raw numbers**
- Small models (logistic regression, random forest, small MLP) want **few, informative inputs**
- A feature = a *summary statistic* that compresses a window into one number
 - `std(x)` $\approx$ "how much does it shake?"
 - `zero_crossings(x)` $\approx$ "how fast does it oscillate?"
- Good features = smaller model = less RAM/flash/latency on the MCU
- Deep learning can learn features from raw data (Day 3, CNN) — at 10–100 $\times$ the resource cost

Be cautious

- [Correlation does not imply causation](#)
- [The importance of domain knowledge](#)



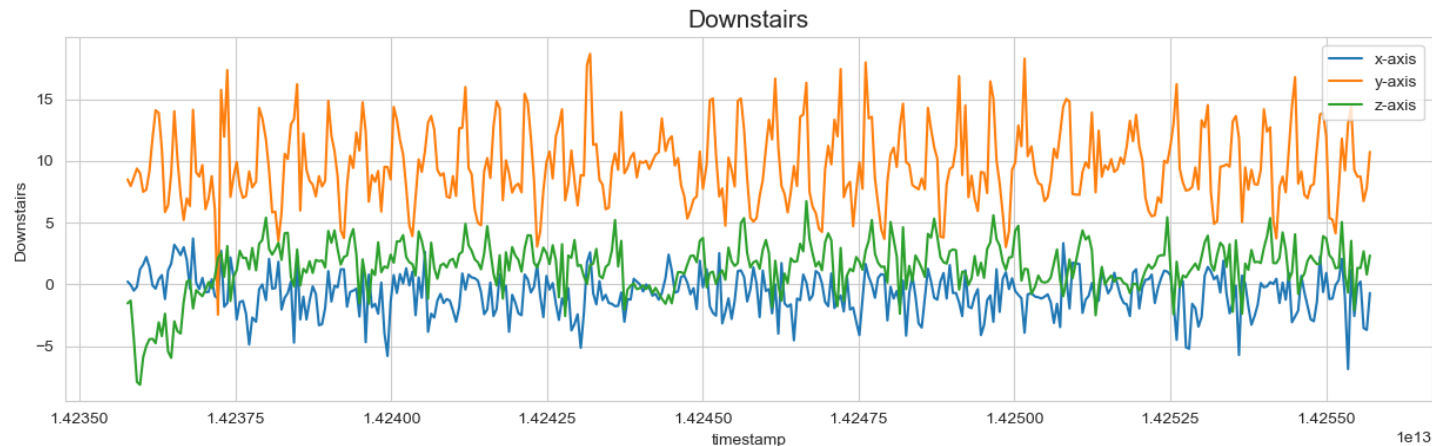
The ML pipeline

sensor → [window] → [FEATURES] → [model] → class
200×3 @ 2s ~10-60 nums RF / NN

- Same features must be computed **twice**, identically:
 - **Training** (Python, on the PC, on the whole dataset)
 - **Inference** (C, on the MCU, on the live window)
- Any mismatch = silent accuracy loss → this module ends with a **Python-vs-C validation**

Windowing recap

- Slice the continuous stream into **fixed-length windows**;
Each window $\rightarrow$ 1 feature Vector $\rightarrow$ 1 prediction
- **Window length**: long enough for several cycles of the phenomenon
 - Fan at ~ 25 Hz rotation $\rightarrow$ 2 s window = ~ 50 revolutions ✓
- **Overlap** (e.g. 50%): more training windows from the same data, smoother live predictions
- Label of a window = the label of its recording



Time-domain features — six useful features for vibration

For each axis (window $x_1 \dots x_N$):

Feature	Formula	Captures
mean	$\bar{x} = \frac{1}{N} \sum x_i$	static orientation / gravity
std	$\sqrt{\frac{1}{N} \sum (x_i - \bar{x})^2}$	vibration intensity
RMS	$\sqrt{\frac{1}{N} \sum x_i^2}$	total energy (incl. DC)
min / max	—	extremes, impacts
zero crossings	# sign changes of $(x_i - \bar{x})$	dominant frequency proxy

The extended zoo (WISDM: 59 features)

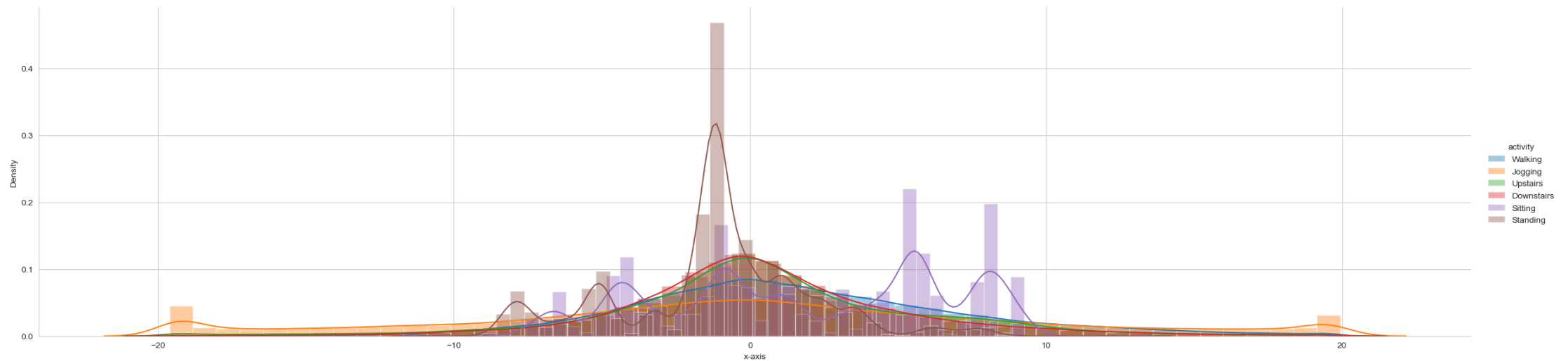
Stage 1 statistical measures from the WISDM walkthrough — per axis:

- mean, std, **average absolute deviation**, min, max, max–min
- median, **median absolute deviation (MAD)**, **IQR**
- neg/pos counts, values-above-mean, **peak count**
- **skewness** (asymmetry), **kurtosis** (spikiness)
- **energy** $\frac{1}{N} \sum x_i^2$, plus cross-axis: **average resultant** $\sqrt{x^2 + y^2 + z^2}$, **SMA**

Stage 3: argmin/argmax indices. → 59 features total.

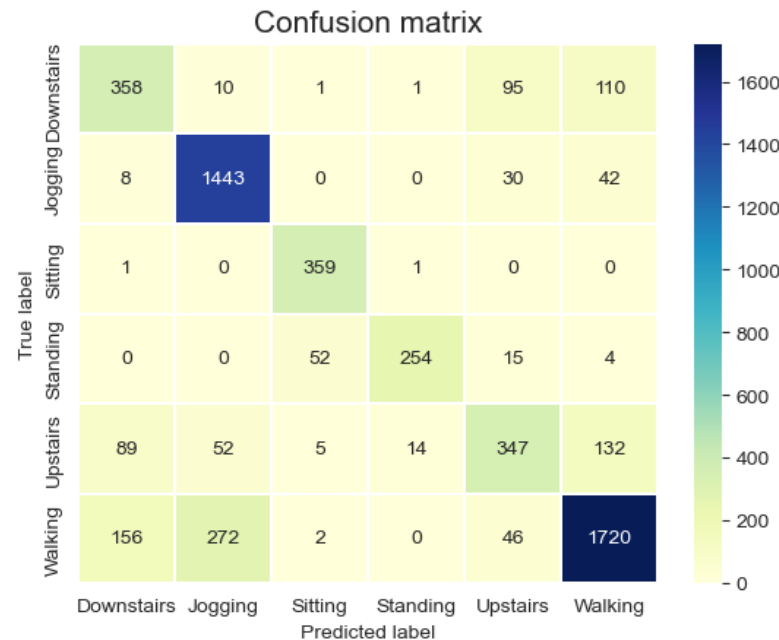
Case study: WISDM human-activity dataset

- 1.07 M accelerometer samples, 20 Hz, 36 users, 6 activities (Walking, Jogging, Up/Downstairs, Sitting, Standing)
- Same shape as our problem: 3-axis accel, multi-class, time-series
- Split **by user** (train: users 1–27, test: 28–36) — the "split along the generalisation axis" rule again



Does it work? Baseline classifier on features

- Logistic regression on 59 features, standardised → **~80 % accuracy** on unseen users
- Reading a **confusion matrix**: rows = true class, columns = predicted, diagonal = correct
- Confusion concentrates where physics says it should: Upstairs ↔ Downstairs ↔ Walking



Random Forest in five minutes (1/2) — the decision tree

- A **decision tree** = a learned cascade of `if (feature < threshold)` splits
- Training = greedily picking, at each node, the split that best separates the classes
- Prediction = follow the branches to a leaf → class

```
if (y_std < 0.021)    → Sitting/Standing  
else if (x_mad < 0.40) → Walking  
else if (z_energy > 12.0) → Jogging  
...
```

No weights, no gradients — just comparisons on your features.

Random Forest in five minutes (2/2) — the forest

- One tree happily **memorises noise** — deep trees overfit
- A **random forest** = many trees, each trained on a *random subset* of samples **and** features → **majority vote**
- Many overfit-but-different trees average out to a robust ensemble
- **Feature importance falls out for free:** every split *uses* a feature and *measurably improves* class purity → sum each feature's contribution over all trees = a ranking

That free ranking is exactly what we use next.

Feature importance — let the forest rank them

```
rf = RandomForestClassifier(n_estimators=15)
rf.fit(X_train, y_train)
pd.DataFrame(rf.feature_importances_, index=X_train.columns) \
    .sort_values('importance', ascending=False).head(10)
```

WISDM result: `y_std`, `y_pos_count`, `y_mad`, `y_IQR`, `y_maxmin_diff`, `x_mad`, ...

- Retrain on **top-10 only**: 79 % → 71 % — 6× fewer features, modest drop
- argmin/argmax indices land at the bottom (importance < 0.003) — dead weight

Feature selection for the MCU

Aim for **~4 time-domain features per axis + band energies** (next section).

Screen candidates with three filters:

1. **Importance** — does the model care? (RF ranking)
2. **Cost** — single pass $O(N)$? needs sort? needs FFT?
3. **Redundancy** — std vs MAD vs IQR: pick *one* spread measure

For the fan lab, a strong starter set:

std , RMS , zero-crossings , kurtosis (time) + band energies (frequency, next)

Where time-domain features fail

Two fan states with **identical RMS**:

- **blocked** : fan spins *faster* (less air load), similar amplitude
- **normal** : nominal RPM

Amplitude statistics are **frequency-blind**. But:

- imbalance lives **at 1× RPM**
- blockage **moves** 1× RPM
- scrape spreads energy **across the spectrum**

→ We need the **frequency domain**.

FFT in five minutes

- DFT: any window = sum of sinusoids; FFT computes it in $O(N \log N)$
- Input: N real samples at $f_s \rightarrow$ output: $N/2$ complex **bins**
- Bin $k \leftrightarrow$ frequency $k \cdot f_s / N$ — resolution = f_s / N
 - 256 samples @ 100 Hz $\rightarrow$ **0.39 Hz/bin**, bins 0–50 Hz
- Magnitude $|X_k|$ = "how much of frequency k ", phase usually discarded
- Multiply by a **window function** (Hann) first to reduce spectral leakage

```
import numpy as np
mag = np.abs(np.fft.rfft(x * np.hanning(len(x)))) / len(x)
freqs = np.fft.rfftfreq(len(x), d=1/fs)
```

Reading the fan spectrum

- **1× RPM fundamental**: fan at 1500 RPM → 25 Hz peak
- **Harmonics**: 2× (50 Hz — at our Nyquist edge!), 3×...
- **Blade-pass**: blades × 1× (7 × 25 = 175 Hz) — *aliased/invisible at 100 Hz sampling*
- Expectations per state:

State	Spectrum
normal	modest 1× peak + low broadband
imbalance	big 1× peak
blocked	1× peak shifted up in frequency
scrape	raised broadband floor, many harmonics
off	flat noise floor

Spectral features: band energy

Cheapest useful spectral feature — sum magnitudes over frequency bands:

$$E_{band} = \sum_{k \in band} |X_k|^2$$

- Our layout at $f_s=100$ Hz, $N=256$: **5 bands × 10 Hz** (0–10, 10–20, 20–30, 30–40, 40–50)
- Imbalance → energy piles into the band containing 1× RPM
- Blockage → energy *moves between* neighbouring bands
- Scrape → all bands rise
- Alternatives: spectral peak (argmax bin → Hz), spectral centroid, peak/total ratio

Aside: this is what EI's Spectral Analysis block does

- Module 4's feature explorer ran: filter → FFT → spectral power in bins + a few statistics
- Same maths, generated for you — plus auto-tuned parameters
- After today you can read the EI DSP block config and know **exactly** what each number is
- ...and you could replace it with your own C code (Module 6/7: exactly that trade-off)



Lab track A — features in Python (~45 min)

[exercises/module5-features/notebooks/](#)

- `01_wisdm_features.ipynb` : the reference walkthrough (59 features → RF → top-10)
- `02_fan_features.ipynb` : **your own fan data** — spectra, band energies, per-class boxplots
- Verify: does the 1×-RPM peak match the RPM you wrote in the metadata?
- **Done when:** you can name the 3 most important features for YOUR dataset

Now in C — the implementation pattern

```
// features.h — one pass, no malloc, no libc beyond sqrtf
typedef struct {
    float mean, std, rms, min, max;
    int zero_crossings;
} time_features_t;

void time_features_compute(const float *x, int n, time_features_t *out);
```

- Accumulate `sum`, `sum_sq`, `min`, `max` in **one loop**
- `std = sqrtf(sum_sq/n - mean*mean)` — population std, matches `numpy.std` default
- Zero crossings: count sign changes of `x[i] - mean` (second short loop)
- **No dynamic allocation** — everything static/stack, MCU-friendly

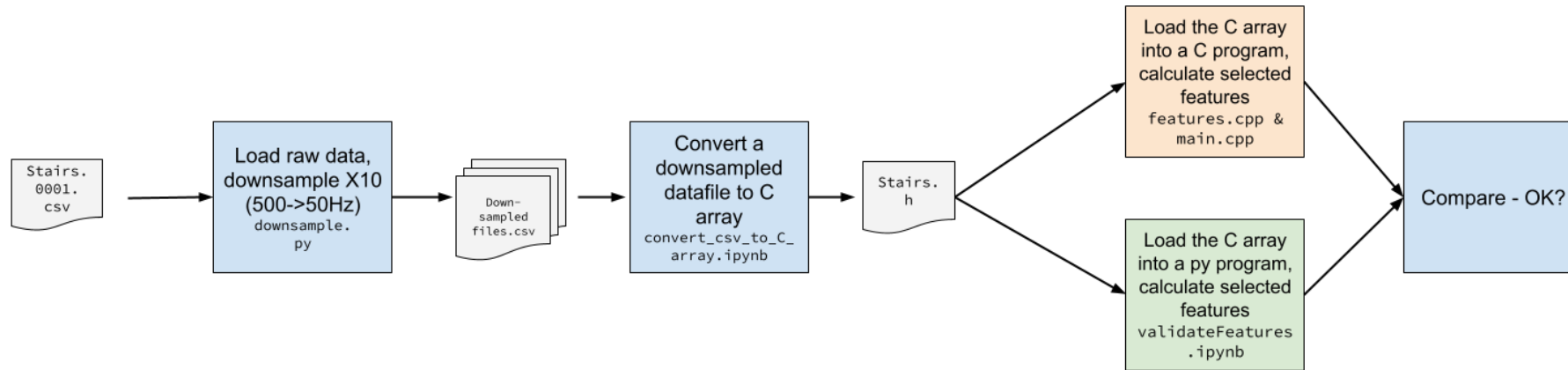
FFT on the nRF52840 — CMSIS-DSP

- Cortex-M4F: hardware FPU + SIMD → ARM's **CMSIS-DSP** library is the native choice

```
#include "arm_math.h"
arm_rfft_fast_instance_f32 S;
arm_rfft_fast_init_f32(&S, 256);
arm_rfft_fast_f32(&S, input, output, 0); // output: interleaved re/im
arm_cmplx_mag_f32(output, mag, 128);    // -> 128 magnitude bins
```

- 256-point float FFT: well under a millisecond at 64 MHz
- CMSIS-DSP also has one-call stats: `arm_mean_f32` , `arm_rms_f32` , `arm_std_f32` —
compare against your loop!

Validation: Python vs C (golden-sample testing)



1. Export a real recorded window (one per class) to a **C array header**
2. Compute features **in C on the device**, print over serial
3. Compute features **in numpy on the same array**
4. Compare: $|py - c| / |py| < 1e-4$ →



Timing your features

```
uint32_t t0 = micros();  
time_features_compute(win_x, N, &fx);  
uint32_t dt = micros() - t0;  
Serial.printf("time features: %lu us\n", dt);
```

- Typical (64 MHz M4F, N=256, per axis): stats $\approx$ tens of μ s, FFT $\approx$ hundreds of μ s
- Compare: window is 2 000 000 μ s long $\rightarrow$ feature cost is $\approx$ **0.1 %** duty cycle
- This headroom is *the* TinyML power story: compute briefly, sleep long



Lab track B — features in C (~75 min)

[exercises/module5-features/features-c/](#)

(track A — the notebooks — ran after the spectral section)

1. Implement mean/std/RMS/min/max/ZC + FFT band energy on the RAK4631
2. Validate: `python validate_features.py --port /dev/ttyACM0` — **golden windows must PASS**
3. Report timing per feature (`micros()`)

Done when: Python and C agree to 6 decimals on the golden window.

Module 5 checkpoint

- ✓
 - You can name, compute and *choose* time-domain features — in numpy and in C
- ✓
 - You found your fan's rotation frequency in its own spectrum
- ✓
 - Band energies separate states that RMS alone cannot
- ✓
 - Your C features match Python to $< 10^{-4}$ relative error — **validated, not hoped**
- ✓
 - You know the compute cost of every feature you plan to deploy

Next (Module 6): frameworks — who turns these features into a deployed model, and what it costs you in licence, footprint and lock-in.